

App Dev Project Report

1. Student Details

- **Name:** Tanisha Chaudhary
- **Roll Number:** 24F3003969
- **Email:** 24f3003969@ds.study.iitm.ac.in
- **About Me:** I am a student at the IIT Madras BS Degree program with a strong interest in Data Science, Machine Learning, and full-stack web development. I enjoy building robust, data-driven applications that solve real-world problems and have aspirations to pursue research in AI.

2. Project Details

- **Project Title:** University Placement Portal
- **Problem Statement:** To design and build a comprehensive, multi-role web application that streamlines the university campus placement process. The portal needs to connect students, recruiting companies, and university administrators to efficiently manage placement drives, track applications, schedule interviews, and generate real-time analytical insights.
- **Approach:** The application was built with a decoupled architecture, utilizing Flask for a robust RESTful API backend and Vue.js for a reactive frontend. It features Role-Based Access Control (RBAC) for Admins, Students, and Companies. To handle heavy operations without freezing the user interface, Celery and Redis were implemented for asynchronous background tasks, such as generating downloadable CSV placement histories and sending automated email notifications.

3. AI/LLM Declaration

- I used Gemini to assist in understanding complex Flask-Celery asynchronous background tasks, styling my pages and using Chart.js and VueQuill.
- The extent of AI/LLM usage is around 10–15%, primarily utilized as a suggestions and bootstrap styling assistance and generating visually attractive application.
- All core application logic, database schema design, frontend component structuring, and system integration were architected and implemented manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework for serving RESTful APIs
Vue.js	Frontend JavaScript framework for building reactive UI components
SQLAlchemy	Object Relational Mapper (ORM) for managing the SQLite database

Technology / Library	Purpose
Celery	Asynchronous task queue for handling background jobs (CSV exports, emails)
Redis	Message broker to facilitate communication between Flask and Celery, and for caching
SQLite	Lightweight local database for storing user, drive, and application data
Chart.js	Frontend visualization library for rendering placement funnel and hiring trend charts
Bootstrap 5	Frontend CSS framework for responsive design and layout structuring
VueQuill	Generating a text editor for Job Description.

5. Database Schema / ER Diagram

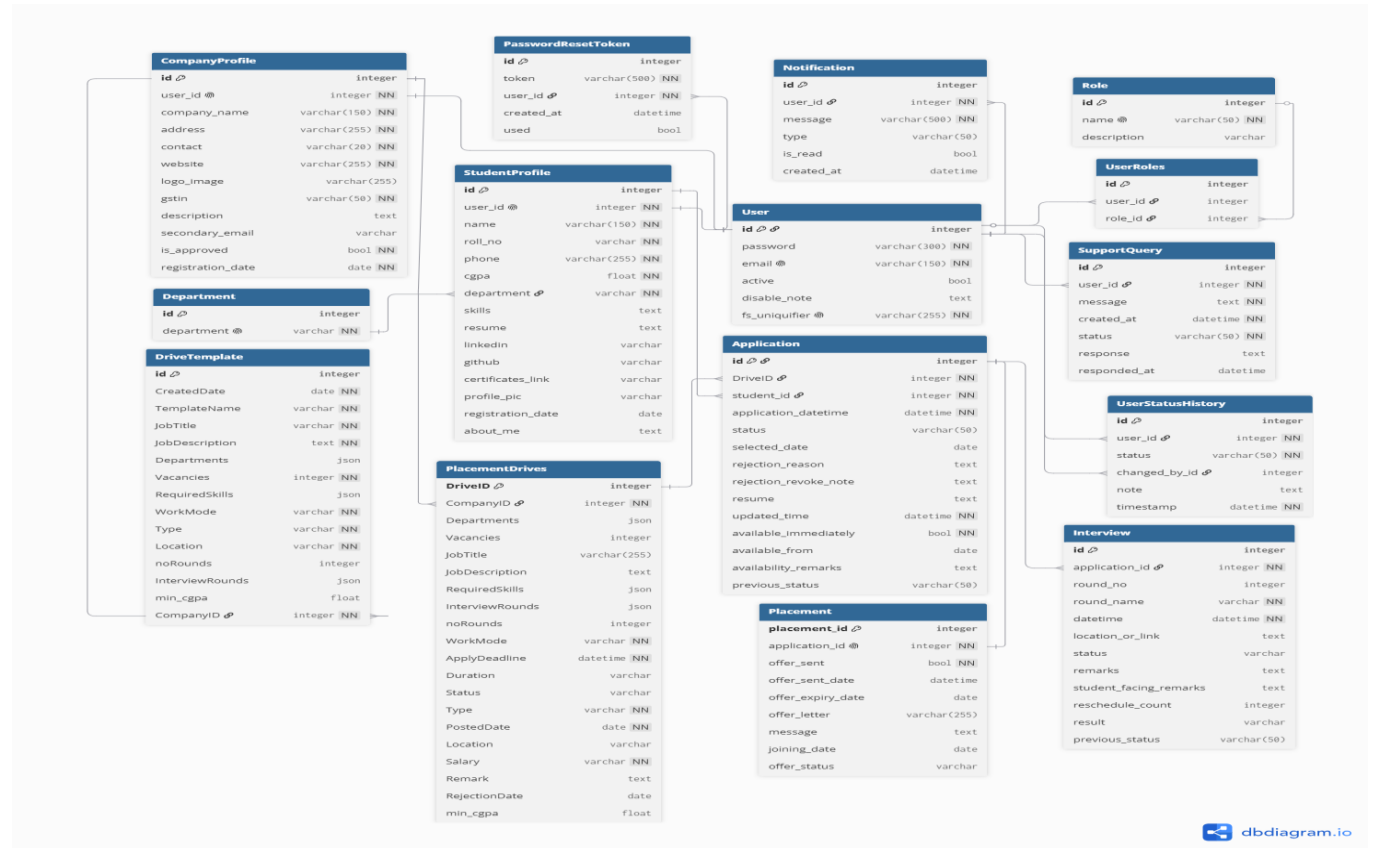
Tables & Architecture

Our application utilizes a highly relational SQLite database comprising 12 tables to manage role-based access, placement workflows, and user data:

- **User:** Core authentication table managing credentials, active status, and security keys.
- **Role & UserRoles:** Manages Role-Based Access Control (RBAC). UserRoles acts as an association table enabling many-to-many relationships between users (Admin, Student, Company) and their permissions.
- **Department:** A lookup table storing valid university departments for data consistency.
- **StudentProfile:** Stores student academics, resumes, and portfolio links. Linked One-to-One with the User table.
- **CompanyProfile:** Stores recruiter details, GSTIN, and company branding. Linked One-to-One with the User table.
- **PlacementDrives and DriveTemplates:** The core entity for job/internship postings, tracking vacancies, deadlines, required skills, and approval statuses which allow us to save our preferences.
- **Application:** Tracks the pivot between a student and a drive. Records application time, resume snapshots, and current selection status.
- **Interview:** Manages multi-round interview scheduling, locations/links, and specific round feedback.
- **Placement:** The final stage entity. Linked One-to-One with Application to track offer letters, joining dates, and offer acceptance statuses.
- **Notifications:** Keep record of in-app notifications for admin, company and student.
- **UserStatusHistory and SupportQuery:** Record of suspensions and queries of students and companies.

Key Relationships:

- **One-to-One:** User ↔ StudentProfile, User ↔ CompanyProfile, Application ↔ Placement.
- **One-to-Many:** CompanyProfile → PlacementDrives, StudentProfile → Application, PlacementDrives → Application, Application → Interview.
- **Many-to-Many:** User ↔ Role (resolved via UserRoles).



6. Some API Resource Endpoints

Endpoint	Method	Description
/api/whoami	GET	Validates the active session token and returns the user's RBAC role (Admin, Student, Comp)
/api/admin_dashboard	GET	Fetches real-time placement statistics and urgent drive queues

Endpoint	Method	Description
<code>/api/whoami</code>	GET	Validates the active session token and returns the user's RBAC role (Admin, Student, Comp)
<code>/api/admin/drives/<id></code>	PUT	Allows admin to approve or reject pending placement drives
<code>/api/admin/export/student/<id></code>	GET	Triggers asynchronous Celery task to export student history
<code>/api/task-status/<id></code>	GET	Polls the server for the completion status of Celery jobs
<code>/api/login</code>	POST	Authenticates user credentials and returns session token

YAML API Definition File:

Included separately in the submission ZIP as [api.yaml](#).

7. Architecture Overview:

- `app.py` – Main Flask application entry point and configuration setup
- `/application/tasks.py` – Contains Celery worker definitions for background processing
- `/static/components/` – Modular Vue.js frontend components (e.g., Dashboards, Profiles)
- `/static/pages/` – Main Vue.js views routed via `router.js`

8. Implemented Features:

- **Role-Based Access Control (RBAC):** Secure, multi-tier authentication and custom dashboards for Administrators, Students, and Company Recruiters.
- **Smart Application Engine:** Automated eligibility validation (CGPA, skills, department) that blocks unqualified applications, alongside a dynamic timeline tracker for students.
- **Asynchronous Processing:** Integrated Celery and Redis to handle heavy background computations—such as generating CSV reports, creating PDF offer letters, and sending automated emails—without blocking the frontend UI.
- **Real-Time Analytics Dashboards:** Interactive data visualizations (Chart.js) displaying placement funnels, department success rates, and algorithmic "Placement Insights" (e.g., skill gap detection).
- **Advanced Administrative Tools:** Centralized "Urgent Action" queues, drive template saving for recruiters, notification tab for all users and comprehensive student history management.

8. Video Presentation

- **Drive Link:** [view video](#)